Lesson 13 Teacher's Key: A Hit That Reacts

GameMaker - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Introducing Alarms - GameMaker's built-in 'do something after N steps' mechanism - genuinely new this week, and necessary because GML can't pause mid-event the way Godot's await or Unity's Coroutine can. This requires splitting the hit-handling logic across TWO events for the first time.

Level 1 - Full Worked Solution

- obj_bullet's Collision event keeps ALL of its existing logic (sound, score, win-check) completely unchanged and immediate - only instance_destroy(other) is replaced with a flash + an alarm.
- obj_enemy's new Alarm 0 event is deliberately tiny - just instance_destroy() - because all the meaningful game-state logic already ran in the Collision event, 6 steps earlier.
- obj_explosion is a fully separate object with its own Create-event alarm and its own Alarm 0 event - it manages its own lifetime with zero involvement from obj_enemy or obj_bullet.

Highlighted lines are new this week; everything else is code you already have.

```
// obj_bullet Collision event with obj_enemy (revised):
audio_play_sound(sfx_hit, 1, false);
instance_create_layer(other.x, other.y, "Instances", obj_explosion);
other.sprite_index = spr_enemy_flash;
// Teacher note: if a student has extended this project with their own
per-enemy color variation via image_blend (e.g. alternating colors in
spawn_enemies()) - not part of this lesson - image_blend is a persistent
instance property, not tied to sprite_index. It keeps multiplying against
whatever sprite is currently showing, so this line alone does NOT make the
flash white for a colored enemy; add other.image_blend = c_white; here
too, or the flash will render as a tinted/lighter version of that enemy's
own spawn color instead of true white.
other.alarm[0] = 6;
obj_gamemanager.enemies_remaining -= 1;
obj_gamemanager.__dnd_score += 10;
if (obj_gamemanager.enemies_remaining <= 0) {
    if (obj_gamemanager.wave >= obj_gamemanager.total_waves) {
        obj_gamemanager.has_won = true;
        obj_gamemanager.game_active = false;
        audio_play_sound(sfx_win, 1, false);
    } else {
        obj_gamemanager.wave += 1;
```

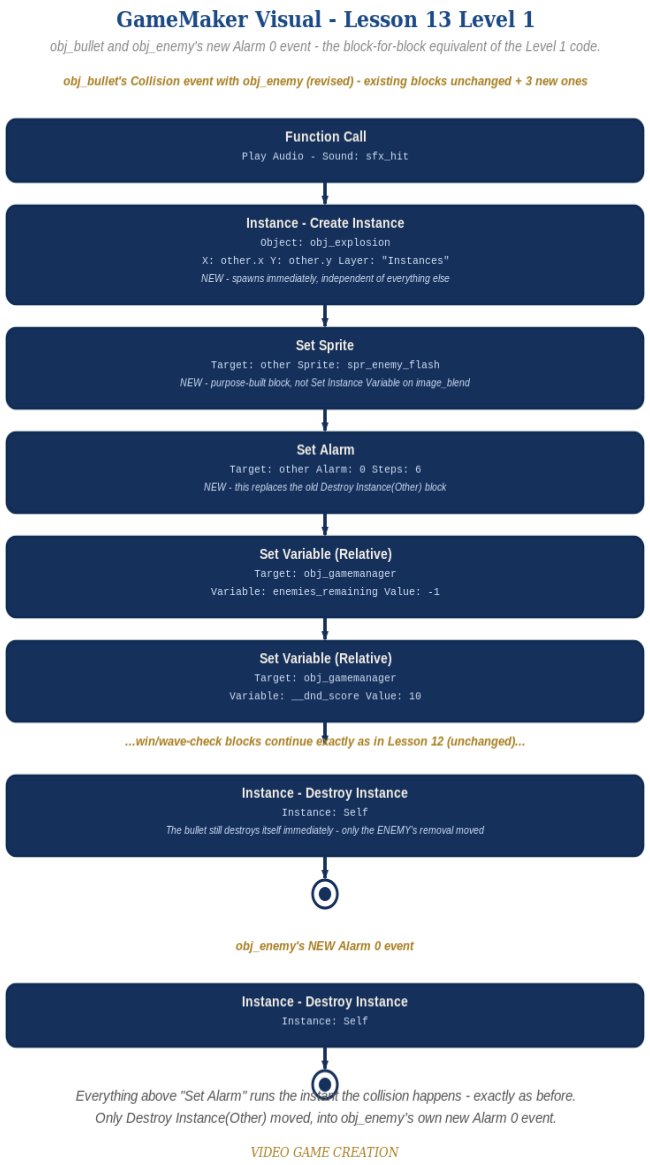
```
        obj_gamemanager.enemies_remaining = 5;
        audio_play_sound(sfx_wave, 1, false);
        spawn_enemies();
    }
}
instance_destroy(self);

// obj_enemy's NEW Alarm 0 event:
instance_destroy();
```

⚠ Some students' projects may have already fixed the "Stalemate" bug from Lessons 10-11 (an optional Level 3 goal), which replaces obj_bullet's inline win/wave-advance if/else with a shared wave_cleared() function called from BOTH obj_bullet's hit path and obj_enemy's escape path. This lesson's explosion/flash/alarm code doesn't change either way - it still goes in obj_bullet's Collision event, right before instance_destroy(other) is removed and replaced with the alarm. What differs is only the win/wave-advance block beneath it: a student with wave_cleared() in place will see a single wave_cleared() call where the Handout's inline nested if/else is shown instead. Check each project's actual structure before walking a student through this week's change.



Everything in the top box happens the instant the hit is detected. Only the enemy's own removal - the very last step - waits for the pause to finish.



obj_bullet's Collision event and obj_enemy's new Alarm 0 event - the block-for-block equivalent of the Level 1 code. (the Visual path)

Why This Structure

- Splitting into two events (Collision starts the flash + alarm, Alarm 0 finishes the removal) is the idiomatic GameMaker way to express 'do this, then after a pause, do that' - GML doesn't support blocking waits inside a single event the way Godot or Unity's languages do.
- Keeping the win-check/score logic in the Collision event (unchanged) rather than moving it into the Alarm 0 event keeps this week's change small and localized - GameMaker students are learning ONE new concept (Alarms), not restructuring everything they already built.
- The flash swaps sprite_index rather than tinting image_blend white, for the same reason Lesson 2's Recolor swaps sprites instead of tinting them: image_blend multiplies against

the sprite's existing color, and multiplying any color by white is a no-op - our enemy sprite is opaque red, so a white image_blend would produce zero visible change.

Level 2 / Level 3 Talking Points

- Ask students to trace exactly which lines moved and which didn't between Lesson 12 and this week - only instance_destroy(other) changed location (into a new event); everything else is untouched.
- For students comparing engines: point out that GameMaker's alarm[0] + Alarm 0 event is conceptually identical to TIC-80's flash-countdown-in-a-table approach this week - both engines lack a blocking wait, so both split 'trigger' and 'finish' into two separate places.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code already produces a
working flash and explosion, so no lines change.
// Checks: play a full round and confirm every single hit flashes and
explodes, with no enemy left stuck mid-flash and no explosion left on
screen;
// confirm score/enemies_remaining/wave-clear/win-check all still happen
at exactly the same instant they did before this lesson - only the enemy's
own final removal is delayed;
// confirm a second bullet hitting an already-flashing enemy does not
double-register a kill.
```

Level 3 - complete code (all three goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// obj_bullet's Collision event is unchanged from Level 1.

// obj_enemy's Step event (NEW) - Level 3 flicker across the existing 6-
step alarm window:
if (alarm[0] > 0 && alarm[0] mod 2 == 0) {
    sprite_index = spr_enemy_flash;
} else if (alarm[0] > 0) {
    sprite_index = spr_enemy_normal;
}

// obj_enemy's Alarm 0 event (unchanged from Level 1):
instance_destroy();

// obj_explosion's Step event (NEW) - Level 3 expanding-burst scale:
image_xscale += 0.08;
image_yscale += 0.08;

// Teacher note: if a student completes this growing-explosion goal, the
recommended
// fix for the "it looks like it's traveling" artifact is to set BOTH
obj_explosion's
// AND obj_enemy's sprite origin to Middle Center (Image Editor > Origin),
not just
// the explosion's. A top-left origin grows asymmetrically (new pixels
only add right
// and down), which reads as drift; centering fixes that correctly rather
than papering
// over it. This is the standard convention for a top-down
```

```
mover/rotator/scaler anyway -
// top-left is only the sprite-import default, kept mainly for UI/HUD
elements and
// tile/background art that are naturally placed by a corner, not a
center.
// Changing the origin shifts what x/y MEAN for that object, so expect to
retune:
//   - spawn_enemies()'s instance_create_layer coordinates (each enemy
will visually
//     jump up-and-left by half its sprite's width/height once its origin
centers);
//   - any escape/boundary check comparing an enemy's y to a screen edge
(it was
//     tuned against the sprite's top edge touching that line, not its
center);
//   - the explosion's own spawn position in obj_bullet's Collision event
- once
//     obj_enemy's origin is centered, other.x/other.y ARE the enemy's
visual center,
//     so any bbox-centering math added to fix the earlier offset can be
removed.
```

Common Student Mistakes

- Leaving instance_destroy(other) in place alongside the new alarm code, causing the enemy to still vanish instantly (the alarm gets set but never has a chance to fire).
- Forgetting to add the new Alarm 0 event to obj_enemy at all - setting alarm[0] does nothing without a matching Alarm event to catch it.
- Accidentally moving score/win-check logic INTO the new Alarm 0 event instead of leaving it in the Collision event - this would delay scoring and win detection by 6 steps, which is unnecessary and inconsistent with every other engine this week.
- If a student has added their own per-enemy color variation via image_blend: leaving image_blend untouched when swapping to spr_enemy_flash, causing the flash to render as a tinted version of that enemy's own spawn color instead of true white - image_blend is a persistent instance property that keeps applying to whatever sprite is current, independent of sprite_index.